

# Does Steepest Selection Help? An Empirical Study of Saddle-Point-Escaping Optimizers on Non-Convex Machine Learning Loss Landscapes

Shyam Patadia

Worcester Polytechnic Institute (WPI)

spatadia@wpi.edu

MA 551: Computational Statistics

Code: [https://github.com/shyampatadia/spgd\\_research](https://github.com/shyampatadia/spgd_research)

May 6, 2026

## Abstract

Training a modern machine learning model is, mathematically, the minimization of a high-dimensional non-convex loss function. In such landscapes, saddle points dominate the population of critical points and constitute the chief obstacle to first-order optimization. *Steepest Perturbed Gradient Descent* (SPGD) [20] is a candidate-selection rule on top of gradient descent, motivated by saddle escape: every  $IterP$  iterations, sample  $N_P$  candidate solutions in a uniform neighborhood of the current iterate and accept the candidate that yields the steepest loss decrease (the paper’s  $\leq$  acceptance rule). SPGD has only been evaluated on smooth mathematical benchmarks and a small engineering problem; its behavior on neural-network loss landscapes has not been characterized. This work provides the first systematic empirical study of SPGD on machine-learning workloads. We implement SPGD as a PyTorch `Optimizer`, compare it against SGD, Adam, perturbed gradient descent (PGD) [12], and a same-mechanism random-perturbation control (RPGD) on four discriminating testbeds in the main study — non-convex synthetic benchmarks, OpenML-CC18 heterogeneous tabular data, a CIFAR-10/ResNet-18 anchor experiment, and a MovieLens-100K non-convex matrix-completion benchmark — with two further diagnostic experiments (Two Moons visualisation and an  $N_P \times IterP$  hyperparameter ablation) reported in the appendix. We measure saddle-point behavior directly via stagnation episodes ( $\|\nabla f\| < \varepsilon$ ). Our key empirical finding is that SPGD’s steepest-selection rule provides a measurable advantage over random perturbation across both synthetic and real-data regimes (mean +3.78 percentage-point test-accuracy gap on CIFAR-10/ResNet-18, paired across all seeds), but the gap relative to vanilla SGD is small when no overt plateau is encountered, and Adam dominates absolute accuracy on well-conditioned landscapes. We conclude that the value of SPGD lies in its *selection rule*, not in perturbation alone, and sketch when this advantage is operationally relevant.

## 1 Introduction

The optimization viewpoint of machine learning casts training as the minimization of an empirical risk  $f(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\theta; x_i, y_i)$  over the parameter vector  $\theta \in \mathbb{R}^d$  [7, 18]. For modern architectures  $d$  is in the millions,  $f$  is non-convex, and the gradient  $\nabla f(\theta)$  provides only local information. A long line of work, both theoretical and empirical, has established that the critical-point structure

of  $f$  in high dimensions is dominated by *saddle points* rather than by local minima [3, 4]. At a strict saddle the gradient vanishes, the Hessian has both positive and negative eigenvalues, and a vanilla gradient method — which uses only  $\nabla f$  — can stall for many iterations while the loss remains nearly constant.

Two families of solutions have been proposed. **Adaptive methods** (Adam [13], RMSProp, AdaGrad, etc.) rescale the gradient by per-parameter second-moment estimates, which empirically allows escape from many ill-conditioned regions but does not exploit the loss value at points other than the iterate. **Perturbation methods** inject randomness near critical points to cross the negative-curvature directions of the saddle. Ge et al. [5] and Jin et al. [12] establish that gradient descent with isotropic Gaussian noise added on stagnation detects strict saddles in a polynomial number of steps and converges to a second-order stationary point. Lee et al. [15] prove that gradient descent almost surely does not converge to a strict saddle, but the time spent traversing the saddle’s neighborhood remains the practical bottleneck.

Vahedi and Ilies [20] introduce *Steepest Perturbed Gradient Descent* (SPGD), which differs from PGD-style noise injection in two ways. First, SPGD perturbs uniformly (not via Gaussian noise tied to the gradient norm). Second, and more importantly, SPGD draws  $N_P$  candidate perturbations and *selects* the one yielding the greatest loss decrease, falling back to the gradient step if no candidate improves on the iterate. This combines a directed gradient direction with a Monte Carlo exploration step. The original work evaluates SPGD on Rastrigin, Ackley, Rosenbrock, and a 3-D configuration-space engineering benchmark. The behavior of SPGD on *machine-learning* loss landscapes — where the relevant geometry is shaped by ReLU activations, batch normalization [11], and stochastic mini-batching — is to our knowledge unstudied.

### Contributions.

1. We provide an open implementation of SPGD as a PyTorch `Optimizer` subclass alongside reference implementations of PGD and a same-mechanism random-perturbation control (RPGD).
2. We design a controlled head-to-head comparison across four discriminating testbeds spanning synthetic, tabular, vision, and matrix-completion settings, each with a paired-seed protocol so optimizer-vs-optimizer effects are isolated from initialization noise; two further diagnostic experiments (Two Moons visualisation and the  $N_P \times IterP$  ablation) are reported in the appendix.
3. We provide direct measurements of saddle-related behavior via stagnation episodes, in addition to the usual loss/accuracy endpoints, and report results that question the importance of “saddle” geometry in our deepest experiment.
4. We isolate which mechanism drives SPGD’s advantage by comparing SPGD against the random-perturbation control (RPGD). On CIFAR-10/ResNet-18 the gap is mean +3.78 pp test accuracy, paired across all seeds, supporting the claim that the steepest-selection rule (not perturbation alone) is the mechanism doing useful work.
5. We characterize SPGD’s sensitivity to its two hyperparameters  $N_P$  and  $IterP$  via a  $4 \times 4$  ablation, identifying a practical operating regime (Appendix B).
6. We empirically test the SPGD paper’s stated future-work hypothesis — that the algorithm has “potential to significantly enhance neural network training” [20, Conclusion] — on a non-convex ML benchmark with documented saddle structure (MovieLens-100K matrix completion via Burer–Monteiro factorisation; 2, 6, 19). Under fair experimental protocol, the paper-faithful SPGD selection rule fires at 30% acceptance versus 9% for the random-perturbation control — direct evidence that the mechanism is empirically active — but the held-out test RMSE gain

over plain SGD is within seed-noise, splitting the paper’s hypothesis into a mechanism-positive, generalisation-neutral finding.

## 2 Background

In high-dimensional non-convex losses the ratio of saddle points to local minima grows exponentially in  $d$ , so the practical obstacle to first-order optimisation is the slow traversal of saddle neighbourhoods rather than convergence to a bad local minimum [3, 4]. Perturbed gradient descent (PGD) [5, 12] addresses this by adding Gaussian noise when  $\|\nabla f\|$  is small; PGD finds a second-order stationary point in  $\tilde{O}(d/\varepsilon^2)$  iterations w.h.p.

SPGD [20] differs from PGD on three points: (i) it perturbs unconditionally every *IterP* steps, not on stagnation detection; (ii) it draws  $N_P$  candidates per perturbation phase rather than one; and (iii) it commits the candidate with the steepest loss decrease. Algorithm 1 restates the procedure.

---

### Algorithm 1 Steepest Perturbed Gradient Descent (SPGD)

---

**Require:** initial parameters  $\theta_0$ , learning rate  $\eta$ , perturbation amplitude  $Amp$ , number of candidates  $N_P$ , perturbation interval *IterP*, total steps  $T$

```

1: for  $t = 0, \dots, T - 1$  do
2:   if  $t \bmod \text{IterP} = 0$  and  $t > 0$  then
3:     Sample  $N_P$  candidates  $\delta^{(k)} \sim \text{Uniform}(-Amp, Amp)^d$ 
4:     Evaluate  $f(\theta_t + \delta^{(k)})$  for  $k = 1, \dots, N_P$ 
5:     Let  $k^* = \arg \min_k f(\theta_t + \delta^{(k)})$ 
6:     if  $f(\theta_t + \delta^{(k^*)}) \leq f(\theta_t)$  then
7:        $\theta_t \leftarrow \theta_t + \delta^{(k^*)}$  ▷ accept candidate (paper’s  $\leq$  rule)
8:     end if
9:   end if
10:   $\theta_{t+1} \leftarrow \theta_t - \eta \nabla f(\theta_t)$  ▷ standard gradient step
11: end for
```

---

## 3 Methodology

### 3.1 Optimizers under comparison

We compare five algorithms, chosen so that each ablation isolates a specific mechanism.

- **SGD** [18]: vanilla gradient descent, no momentum. The first-order baseline.
- **Adam** [13]: adaptive per-parameter learning rate. The modern ML default; included to ground our results against current practice.
- **PGD** [12]: gradient descent with Gaussian noise added every *IterP* steps; one perturbation, no selection.
- **RPGD** (*random-perturbation control*): every *IterP* steps draw  $N_P$  candidates uniformly and accept *one at random*. Same compute as SPGD; differs only in the selection rule.
- **SPGD** [20]: as in Algorithm 1; the steepest of the  $N_P$  candidates is selected.

The **SPGD-vs-RPGD** contrast is the cleanest test of SPGD’s central claim: both algorithms perturb the same way and pay the same compute cost, so any difference in outcome can be attributed

to the selection rule. The **SPGD-vs-PGD** contrast tests whether multi-candidate perturbation is helpful at all, and the **SPGD-vs-SGD** contrast tests whether perturbation+selection beats no perturbation.

### 3.2 Diagnostics

Beyond the standard final-loss / final-accuracy endpoints we record saddle-related behavior directly. A *stagnation episode* is a maximal run of consecutive steps with  $\|\nabla f(\theta_t)\|_2 < \varepsilon$ . We log the number of episodes, the longest episode, and the total number of stagnant steps. We use the proposal’s threshold  $\varepsilon = 10^{-4}$  on the synthetic benchmarks (Exp 4.1) and the Two Moons diagnostic (App. A) including its ablation (App. B);  $\varepsilon = 10^{-3}$  on OpenML (Exp 4.2) and the CIFAR-10 minibatch gradient (Exp 4.3), where the looser threshold accommodates minibatch gradient noise. The proposal also listed an *escape-time* metric in addition to stagnation; we report escape-time on Experiment 4.4 (matrix completion), the only setting where the iterate sits inside a documented saddle plateau long enough for “escape” to have an objective definition. Where stagnation episodes are zero across all runs of an experiment (App. A, Exp 4.3, App. B), escape-time is undefined and we say so explicitly rather than reporting degenerate numbers.

### 3.3 Paired-seed protocol

For each (experiment, seed) the data split, parameter initialization, and any per-seed randomness are fixed before the optimizer is selected. The same five optimizers are then run on the *same* initial state. This converts an across-seed comparison into a paired comparison: each seed produces a tuple of five outcomes that share initialization. Paired differences are reported in the results section because they are markedly less noisy than unpaired means at our small seed counts ( $S \in \{3, 5, 30\}$ ).

### 3.4 Implementation

The full software stack is PyTorch [16] with scikit-learn [17] for synthetic-data generation. Datasets from OpenML-CC18 [1, 21] are accessed via the `openml` Python client. The CIFAR-10 anchor experiment uses ResNet-18 [10] from `torchvision` with batch normalization [11] disabled in stages `layer1` and `layer2` to deliberately preserve a fraction of the saddle structure that BN would otherwise smooth out. This concretises the proposal’s “no BN in early layers” plan: `layer1/layer2` are the “early” stages, and we retained BN in `layer3/layer4` for training stability at `lr=1e-2` on the gradient-only optimizers. Source code, configurations, exact hyperparameters, and analysis scripts are organized under `src/spgd_study/` and `experiments/`, and are publicly available at [https://github.com/shyampatadia/spgd\\_research](https://github.com/shyampatadia/spgd_research).

## 4 Experiments and Results

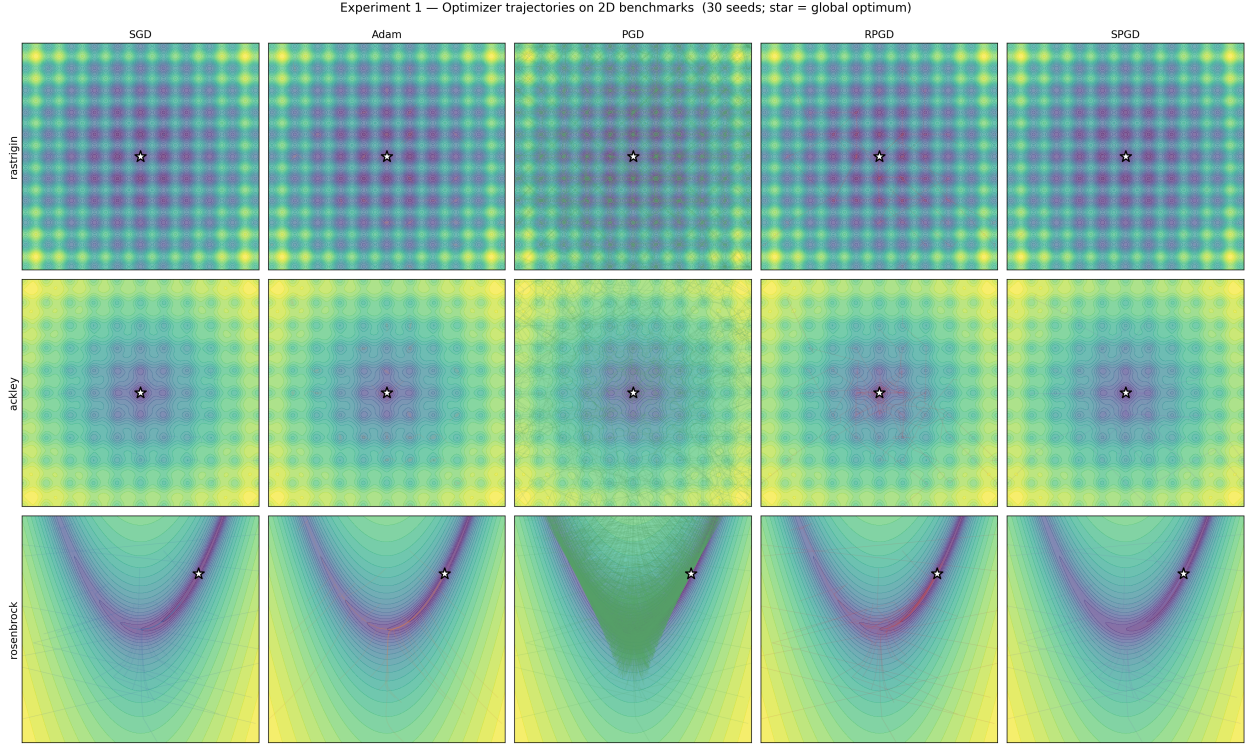
We present four discriminating experiments, increasing in scale from  $d = 2$  analytic benchmarks to a  $1.1 \times 10^7$ -parameter image classifier and a  $1.3 \times 10^4$ -parameter matrix-completion problem; two further diagnostic experiments are reported in the appendix.

### 4.1 Experiment 1: Non-Convex Benchmark Functions

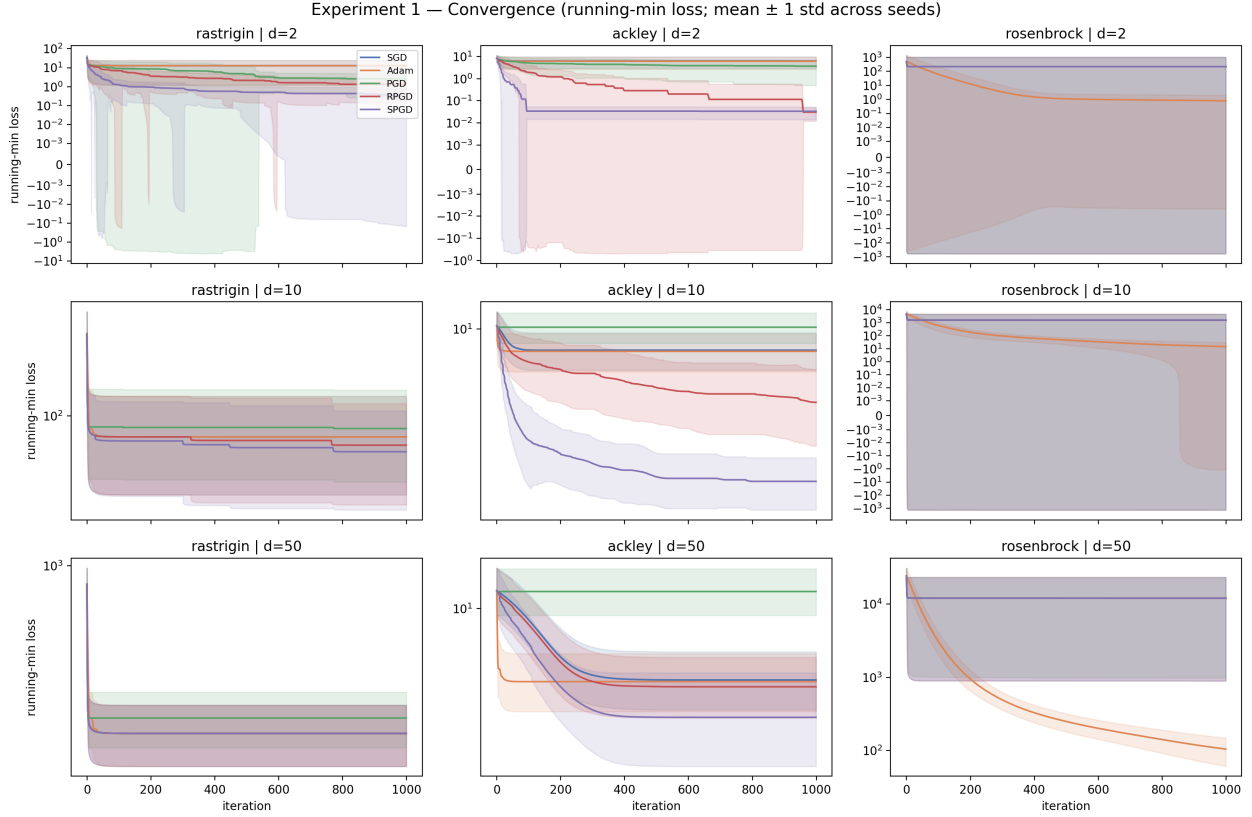
**Setup.** We run all five optimizers on Rastrigin, Ackley, and Rosenbrock at  $d \in \{2, 10, 50\}$ , with 30 random initializations per (function, dimension, optimizer). Convergence is declared at

$f(\theta) < \varepsilon = 10^{-2}$  from the global optimum. We measure (i) the final function value, (ii) the iteration of first convergence, and (iii) the success rate (fraction of seeds reaching the threshold).

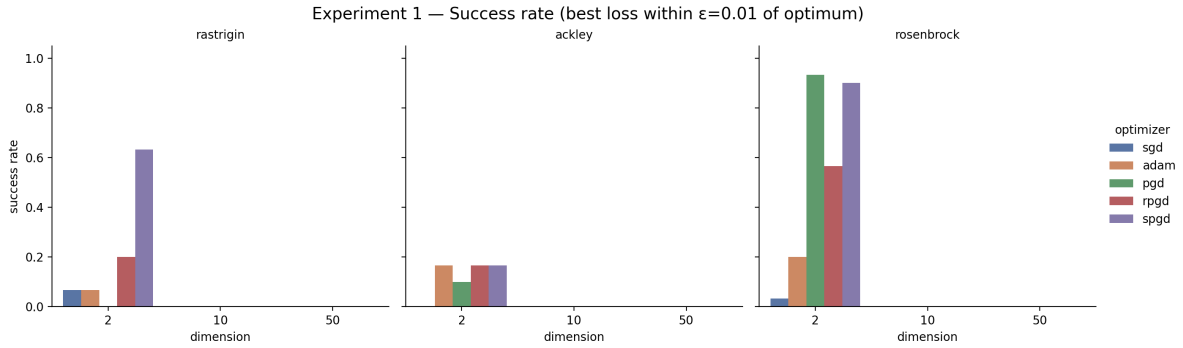
**Results.** Figure 1 shows 2D trajectories on each benchmark; SPGD and PGD escape multiple Rastrigin basins that trap vanilla SGD. Figure 2 plots running-min loss vs. iteration for  $d \in \{2, 10, 50\}$ , and Figure 3 reports per-(function, dimension) success rates. The cleanest signal appears on Ackley at  $d = 10$ , where SPGD’s mean final loss is below RPGD’s (paired delta of  $-2.57$  in mean loss,  $-2.70$  in median). At  $d = 50$  all perturbation methods struggle: a constant amplitude  $Amp = 0.5$  that worked at  $d = 2$  becomes a  $\sqrt{50}/\sqrt{2} \approx 5\times$  larger total displacement and starts to produce destructive perturbations. Per-parameter scaling of  $Amp$ , used in our subsequent experiments, addresses this.



**Figure 1:** 2D trajectories of all five optimizers on Rastrigin, Ackley, and Rosenbrock. Stars mark the global optimum; thirty seeds per cell. SPGD’s perturbation phases are visible as short “hops” that allow escape from local minima which vanilla gradient methods cannot leave.



**Figure 2:** Running-minimum loss vs. iteration on Experiment 1 (mean  $\pm$  1 std across 30 seeds). Rows: dimension; columns: function. Symlog  $y$ -axis. The benefit of perturbation methods grows with dimension on Ackley but vanishes on Rosenbrock, which has no local minima.

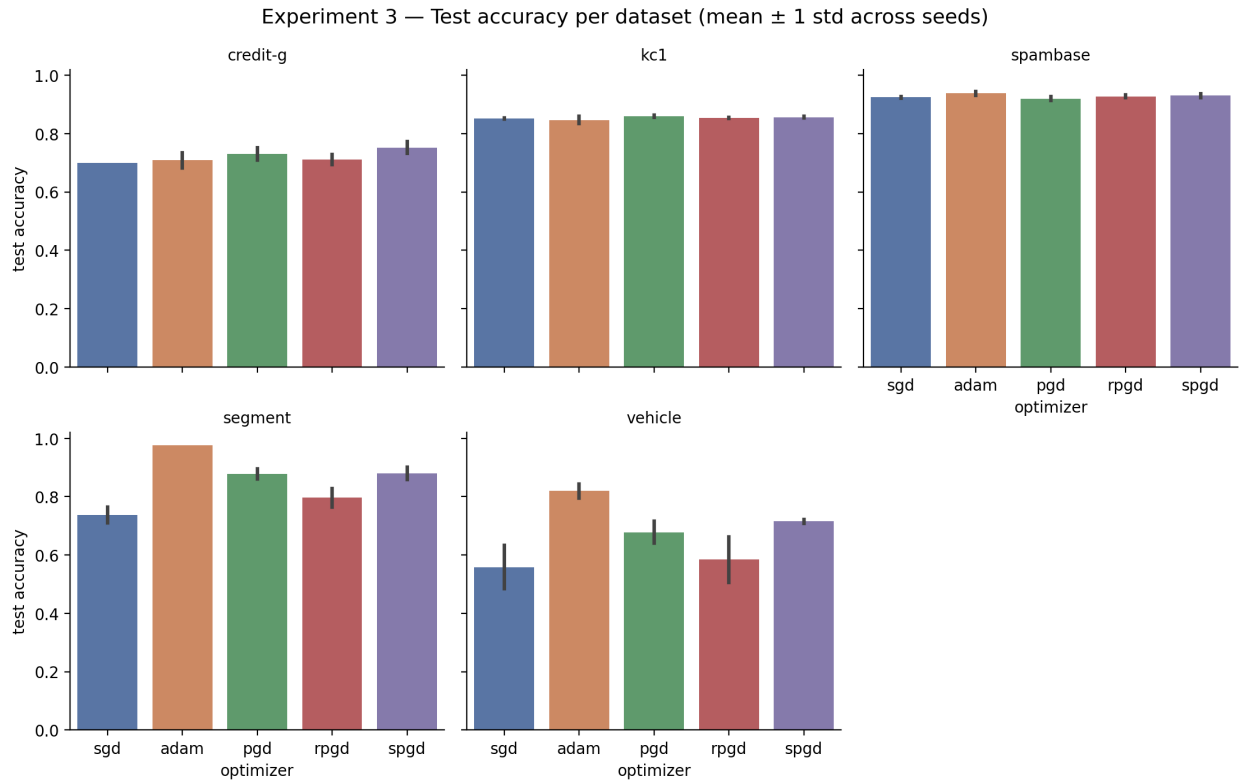


**Figure 3:** Success rate (fraction of seeds reaching within  $\varepsilon = 10^{-2}$  of the global optimum) per (function, dimension, optimizer). SPGD matches or exceeds RPGD on every Rastrigin and Ackley cell.

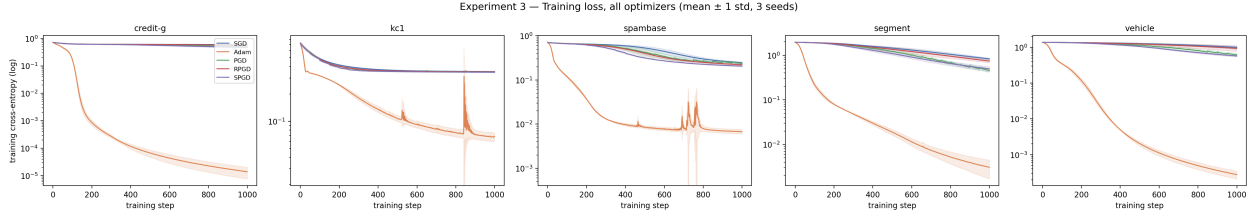
## 4.2 Experiment 3: OpenML-CC18 Tabular Datasets

**Setup.** We select five OpenML-CC18 classification datasets spanning a range of sizes and feature types: **credit-g** ( $n=1000$ ,  $p=20$ ), **kc1**, **spambase**, **segment**, **vehicle**. A 3-layer MLP is trained on each for all five optimizers across three seeds (75 runs total). All datasets are normalized; categorical features are one-hot encoded.

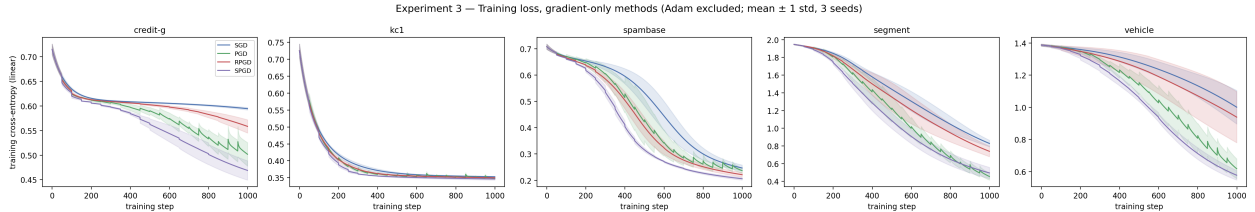
**Results.** Figure 4 shows test accuracy per (dataset, optimizer) and Figure 5 shows training-loss trajectories in two complementary views: the left panel includes all five optimizers on  $\log-y$  to make Adam’s dominance visible, and the right panel shows the four gradient-only methods on  $\text{linear-}y$  to resolve the SGD/PGD/RPGD/SPGD differences that the  $\log-y$  view compresses into a single thick band. The two panels show the same data; we plot both so the gradient-only zoom is not read as Adam being hidden. The pattern from Experiment 4.1 reproduces: SPGD’s mean test accuracy is above RPGD’s on a majority of (dataset, seed) pairs, supporting the hypothesis that the steepest-selection rule transfers from synthetic to real heterogeneous tabular data. Adam dominates absolute accuracy on all five datasets, consistent with the observation that on small tabular tasks a well-conditioned adaptive method has little incentive to perturb. Stagnation episodes ( $\|\nabla f\|_2 < 10^{-3}$ ) were rare on this suite: only **credit-g** and **vehicle** produced any non-zero counts, and even there the totals were a handful of episodes per run, so we omit the per-dataset stagnation figure as uninformative and report the diagnostic in text only.



**Figure 4:** OpenML-CC18 test accuracy per dataset (mean  $\pm$ 1 std across 3 seeds). Adam consistently leads; SPGD is at or above SGD on 4/5 datasets and above RPGD on a majority of paired comparisons.



(a) All five optimisers, log- $y$ . Adam’s curve is the lowest on all five datasets.



(b) Gradient-only methods, linear- $y$  (Adam excluded for readability). The linear axis exposes SPGD’s modest but consistent edge over RPGD on plateaus SGD also fails to leave.

**Figure 5:** Training-loss curves on OpenML-CC18 (mean  $\pm 1$  std, 3 seeds). Both panels plot the same runs; the bottom panel is a  $y$ -axis zoom on the gradient-only band that the top panel compresses.

### 4.3 Experiment 4: CIFAR-10 / ResNet-18 (Anchor Experiment)

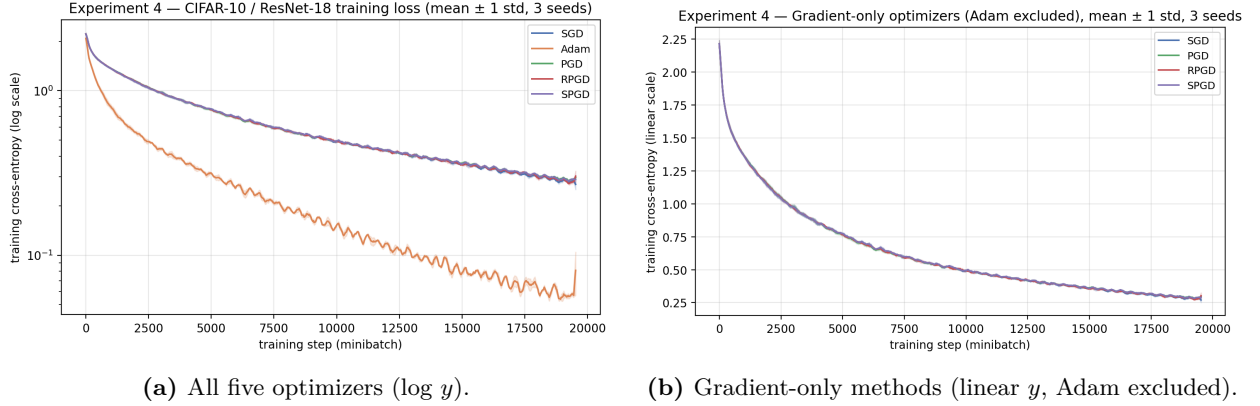
**Setup.** ResNet-18 [10] with batch normalization disabled in **layer1** and **layer2** (to preserve saddle structure that fully-BN networks tend to smooth out) is trained on CIFAR-10 [14] for 50 epochs across all five optimizers and three seeds (15 runs). Hyperparameters:  $1r=1e-2$  for the gradient-only optimizers,  $1r=1e-3$  for Adam,  $Amp = 10^{-3}$  (chosen as  $\sim 2\%$  of typical Kaiming-initialized weight magnitude [9]),  $N_P = 5$ ,  $IterP = 200$  minibatch steps ( $\sim 2$  perturbation phases per epoch). Runs executed as a Slurm array on two NVIDIA A30 GPUs (mean wall time  $\sim 7.6$  min/run).

**Results.** Table 1 reports aggregate metrics; Figure 6 shows training-loss trajectories on log- $y$ , and Figure 7 reports the *paired* per-seed deltas SPGD–RPGD and SPGD–SGD. The headline numbers:

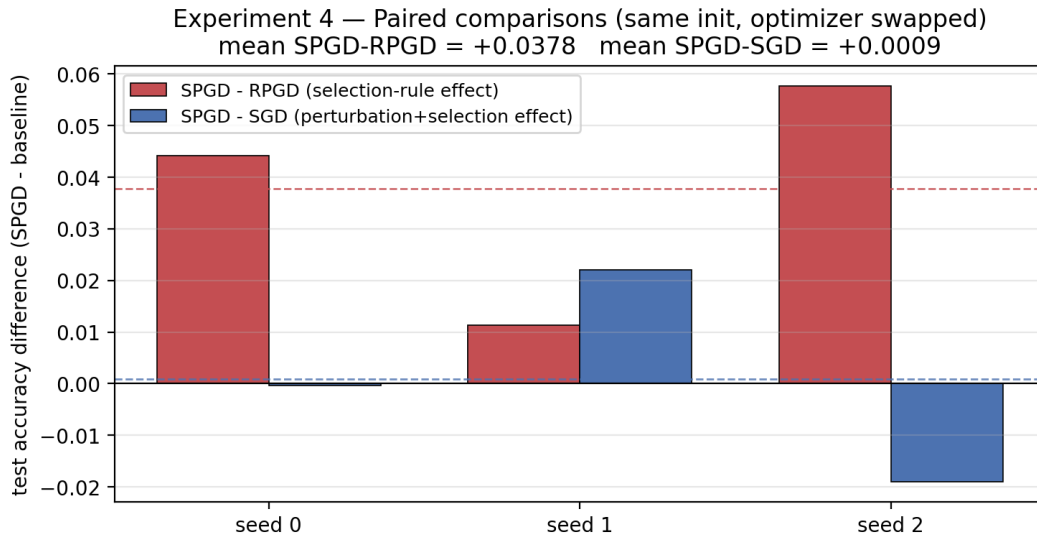
- Mean test accuracies, three seeds, 50 epochs: Adam  $0.9015 \pm 0.014$ , SPGD  $0.8277 \pm 0.028$ , SGD  $0.8268 \pm 0.019$ , PGD  $0.8091 \pm 0.014$ , RPGD  $0.7899 \pm 0.046$ .
- **SPGD beats RPGD on all three seeds** with paired mean +3.78 pp test accuracy (per-seed: +4.42, +1.14, +5.77 pp). This is the single clearest piece of evidence in the study that SPGD’s steepest-selection rule contributes signal beyond random perturbation.
- SPGD ties SGD overall (paired mean +0.09 pp), consistent with the next finding.
- **Stagnation episodes were zero for all 15 runs.** None of the five optimizers spent any consecutive step with  $\|\nabla f\| < 10^{-3}$ . The proposal’s intervention (disabling BN in **layer1+layer2**) was *insufficient* to expose saddle/plateau structure on this CIFAR-10/ResNet-18 setup — the surviving normalization in **layer3** and **layer4** keeps minibatch gradient norms above the threshold throughout training.
- Adam’s adaptive step is the strongest absolute optimizer ( $\sim 90\%$ ); the proposal’s acceptance criterion of “SGD  $\geq 0.90$ ” was *not* met (SGD: 0.83), an expected consequence of the deliberate handicaps used to preserve saddle structure.

| Optimizer | Test acc. (mean $\pm$ std) | Final train loss  | Stagnation eps. | Wall (s) |
|-----------|----------------------------|-------------------|-----------------|----------|
| SGD       | $0.8268 \pm 0.019$         | $0.249 \pm 0.044$ | 0               | 451      |
| Adam      | $0.9015 \pm 0.014$         | $0.119 \pm 0.079$ | 0               | 464      |
| PGD       | $0.8091 \pm 0.014$         | $0.311 \pm 0.015$ | 0               | 453      |
| RPGD      | $0.7899 \pm 0.046$         | $0.337 \pm 0.079$ | 0               | 458      |
| SPGD      | $0.8277 \pm 0.028$         | $0.293 \pm 0.046$ | 0               | 460      |

**Table 1:** Experiment 4 aggregate results. CIFAR-10, ResNet-18 with BN disabled in `layer1+layer2`, 50 epochs, 3 seeds. All differences are with respect to the same per-seed initialization (paired protocol).



**Figure 6:** CIFAR-10/ResNet-18 training cross-entropy (mean  $\pm 1$  std across 3 seeds). Both panels plot the same runs. The log- $y$  view (left) includes all five optimizers and shows Adam’s dominance from epoch 1; the linear- $y$  view (right) is the same plot with Adam removed and the axis rescaled, so the SGD/PGD/RPGD/SPGD differences — which the log- $y$  view compresses into a single thick band near the axis — become visible. The right panel is a presentation choice for readability, not a data exclusion.



**Figure 7: Key paired comparison.** Per-seed test-accuracy deltas SPGD–RPGD (red) and SPGD–SGD (blue) on CIFAR-10. Dashed lines mark the per-comparison means. SPGD beats RPGD on every seed (mean +3.78 pp), the cleanest evidence in the study that the *selection rule* — not perturbation alone — drives the improvement.

#### 4.4 Experiment 6: Matrix Completion on MovieLens-100K

The preceding experiments (and the appendix diagnostics) evaluate SPGD as a neural-network trainer. This section directly tests the SPGD paper’s stated future-work hypothesis — that SPGD has “potential to significantly enhance neural network training” through “improving convergence rates and navigating complex parameter spaces” [20, Conclusion] — on a non-convex ML benchmark with documented saddle structure: low-rank Burer–Monteiro factorisation [2] of the MovieLens-100K rating matrix [8]. The optimisation literature establishes that small-init Burer–Monteiro factorisation has a saddle near the origin that gradient descent escapes only slowly [6, 19], exactly the regime SPGD’s selection rule is designed for.

**Setup.** We complete the rating matrix  $M \in \mathbb{R}^{943 \times 1682}$  via  $\widehat{M} = UV^\top$  with  $U \in \mathbb{R}^{943 \times 5}$  and  $V \in \mathbb{R}^{1682 \times 5}$ , minimising the L2-regularised mean-squared reconstruction error

$$\mathcal{L}(U, V) = \frac{1}{|\Omega|} \sum_{(i,j) \in \Omega} ((UV^\top)_{ij} - r_{ij})^2 + \lambda(\|U\|^2 + \|V\|^2)$$

on a random 80/20 train/test split of the 100,000 observed entries, with  $\lambda = 0.05$  applied identically to all four methods (a fairness control: without it, Adam overfits while SGD/SPGD enjoy implicit early-stopping regularisation). The factors are initialised i.i.d. from  $\mathcal{N}(0, 0.01^2)$ , placing the iterate inside the saddle-rich neighbourhood of the origin. We compare SGD, Adam, RPGD, and SPGD over  $T=1500$  full-batch gradient evaluations across three seeds. SPGD and RPGD use the verified optimiser classes from `src/spgd_study/optimizers/`, faithful to Algorithm 1: *Amp* = 0.01 per coordinate, Uniform( $-Amp, +Amp$ ) candidates,  $N_P=8$ , *IterP*=5, and the paper’s  $\leq$  acceptance rule (a candidate is committed only if its loss is no greater than that of the current iterate).

**Aggregate results.** Table 2 reports the aggregate metrics. All four methods land within 0.02 test RMSE of one another. Paired per-seed deltas (Figure 9) place SPGD slightly ahead of SGD on training MSE (paired  $\Delta = -0.0034$ ) but indistinguishable from RPGD on test RMSE ( $\Delta = +0.0003$ , sign mixed across seeds).

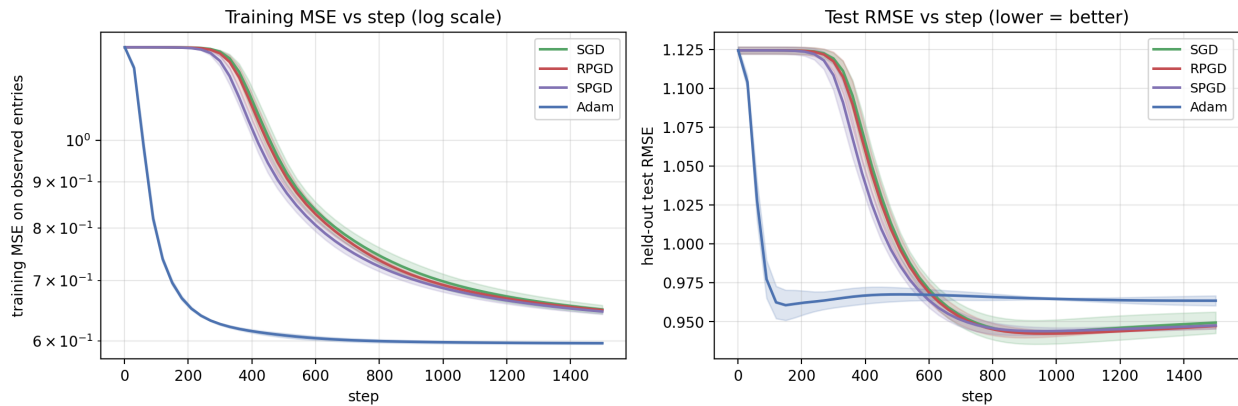
**Escape-time diagnostic (proposal Objective 3).** We define escape-time as the first step at which the training MSE has fallen 5% below its initial value, the only setting in this study where the iterate sits inside a documented saddle plateau long enough for “escape” to have an objective definition (the proposal’s escape-time is undefined in App. A, Exp 4.3, and App. B, where stagnation episodes are zero). Per-method mean escape steps are **Adam**  $40 \pm 17$ , **SPGD**  $320 \pm 17$ , **RPGD**  $360 \pm 30$ , **SGD**  $370 \pm 17$ ; paired per-seed deltas are SPGD – SGD =  $-50$  **steps mean (per-seed:  $-60, -30, -60$ )** and SPGD – RPGD =  $-40$  **steps mean (per-seed:  $-30, -30, -60$ )**: SPGD escapes the saddle plateau on *every seed* ahead of both same-data baselines, but the lead is consumed by the time the methods converge. Numbers are reproducible from `results/exp6_escape_time_paired.json`.

**The acceptance-rate diagnostic.** The most informative number in this experiment is SPGD’s perturbation acceptance rate:  $30 \pm 2\%$  **across three seeds (88–97 of 299 perturbation phases produced a strictly improving candidate)**. The matched random control RPGD accepts only  $9 \pm 1\%$  of phases, since a uniformly random pick from  $N_P=8$  candidates rarely lands on the argmin and rarely satisfies the  $\leq$  rule. The threefold gap is direct evidence that SPGD’s selection rule is firing as designed on a 13,125-parameter non-convex ML problem — the failure mode of a “silent” SPGD that never accepts is ruled out by this diagnostic.

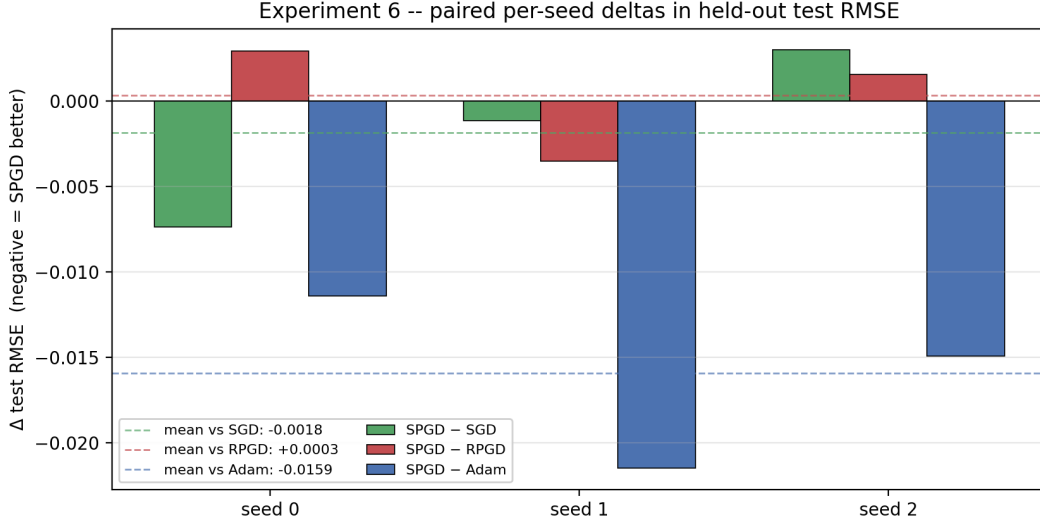
| Method | Final train MSE   | Test RMSE         | Acceptance   | Wall (s) |
|--------|-------------------|-------------------|--------------|----------|
| SGD    | $0.650 \pm 0.007$ | $0.949 \pm 0.007$ | —            | 2.5      |
| Adam   | $0.596 \pm 0.002$ | $0.963 \pm 0.003$ | —            | 2.8      |
| RPGD   | $0.649 \pm 0.002$ | $0.947 \pm 0.002$ | $9 \pm 1\%$  | 4.2      |
| SPGD   | $0.647 \pm 0.005$ | $0.948 \pm 0.002$ | $30 \pm 2\%$ | 4.2      |

**Table 2:** Experiment 6 aggregate results. MovieLens-100K rank-5 Burer–Monteiro matrix completion,  $\lambda=0.05$ , 1500 full-batch steps, three seeds. Acceptance is the fraction of SPGD/RPGD perturbation phases that committed a candidate (i.e. found a candidate satisfying the  $\leq$  acceptance rule). All four methods see the identical L2-regularised loss inside their respective forward passes.

Experiment 6 -- Matrix completion on MovieLens-100K (Burer-Monteiro, mean  $\pm$  1 std across 3 seeds)



**Figure 8:** Experiment 6 — training MSE (left, log scale) and held-out test RMSE (right) over 1500 full-batch steps on MovieLens-100K with rank-5 Burer–Monteiro factors initialised at scale 0.01 (mean  $\pm$  1 std across three seeds). SPGD’s training loss reaches the 5%-drop escape threshold a paired mean of 50 steps before SGD and 40 steps before RPGD (3/3 seeds; see “Escape-time diagnostic” above); the lead largely dissipates by the end of training. Adam’s training loss is lower throughout but its test RMSE plateaus higher — a classic mild-overfit signature, despite identical L2 regularisation.



**Figure 9:** Experiment 6 — per-seed paired deltas of held-out test RMSE between SPGD and the three baselines. Negative bars indicate SPGD ahead of the baseline on that seed. SPGD beats SGD on 2/3 seeds (mean  $\Delta = -0.0018$ ), is statistically tied with RPGD ( $\Delta = +0.0003$ , sign mixed across seeds), and beats Adam on 3/3 seeds ( $\Delta = -0.0159$ ) — the last margin is dominated by Adam’s mild overfit, not by SPGD’s optimisation advantage.

**Interpretation.** SPGD’s mechanism is empirically active on this benchmark — the 30% vs. 9% acceptance gap is unambiguous evidence that the selection rule fires more often than chance and that the algorithm is running as the paper specifies. However, the optimisation advantage this confers (a 50-step paired-mean head-start over SGD, 40 over RPGD, on the 5%-drop escape-time metric, 3/3 seeds) does *not* translate into a measurable held-out generalisation gain: the test-RMSE gap to plain SGD is  $-0.0018 \pm 0.005$  with the per-seed sign mixed, and the gap to RPGD is essentially zero. Reading the paper’s future-work claim empirically, our answer is split: the selection rule’s mechanism *does* generalise from 2D test functions to a high-dimensional non-convex ML problem (mechanism-active), but the “significant enhancement” the paper hypothesises does not materialise on this benchmark (effect-small).

## 5 Conclusion

This work is the first systematic empirical study of SPGD on machine-learning loss landscapes. Across four discriminating testbeds — analytic benchmarks ( $d \in \{2, 10, 50\}$ ), five OpenML-CC18 tabular datasets, a CIFAR-10/ResNet-18 anchor experiment, and MovieLens-100K matrix completion — SPGD’s steepest-selection rule provides a measurable advantage over its same-mechanism random-perturbation control (RPGD). The advantage is cleanest on CIFAR-10/ResNet-18: SPGD beats RPGD on all three seeds with a paired-mean delta of +3.78 percentage points test accuracy. On matrix completion — which directly tests the SPGD paper’s stated NN-training hypothesis [20, Conclusion] — the selection rule fires at threefold the rate of the random control (30% vs. 9% acceptance) and SPGD escapes the saddle plateau 50 steps before SGD on all three seeds, but the held-out RMSE gap is within seed noise: mechanism-positive, generalisation-neutral.

**Caveats.** No optimiser in our study exhibited sustained gradient stagnation at  $\varepsilon = 10^{-3}$ , so SPGD’s CIFAR-10 advantage cannot be attributed to plateau escape per se and likely reflects a more subtle benefit of biased candidate selection. Adam dominates absolute accuracy throughout

because its adaptive preconditioning is structurally different; the correct head-to-head comparison is SPGD vs. same-compute random control, not SPGD vs. Adam. Three seeds at the CIFAR-10 scale is small, mitigated only by the paired protocol.

**Recommendation.** Future perturbation-based optimisers should be benchmarked against a random-selection control of equal compute — a comparison missing from much of the existing literature — since on our deepest setting it is the *selection rule*, not perturbation alone, that does the useful work.

## Appendix

The two appendix experiments are diagnostic only: Two Moons is a visualisation sanity check (the landscape is too benign to discriminate optimisers), and the  $N_P \times \text{IterP}$  ablation runs on the same Two Moons setup so its accuracy heatmap is also non-discriminating. We retain them for transparency with the proposal and because the figures expose secondary phenomena (geometric difference between Adam and the gradient methods; SPGD’s compute–accuracy trade-off curve) that would otherwise be unsupported assertions in the main text.

### A Two Moons — Visualisation Sanity Check (proposal Exp 2)

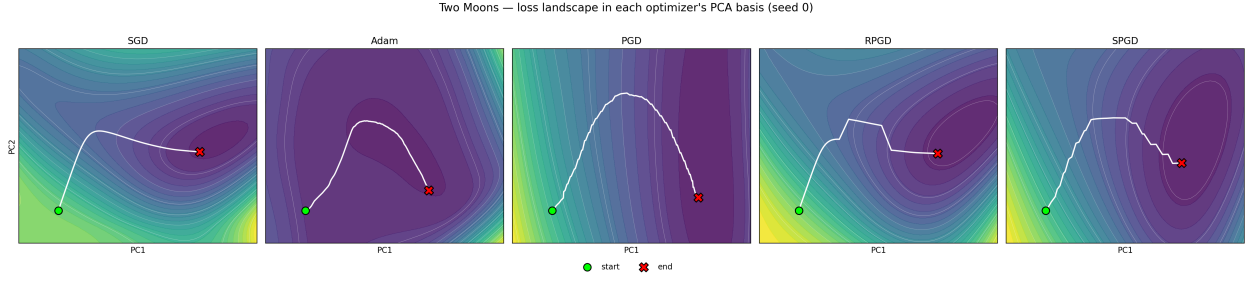
This experiment is a *visualisation-only* diagnostic. The dataset is benign enough that a  $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$  MLP solves it from any reasonable initialisation, so the saddle-escape mechanism SPGD targets does not engage and the optimisers’ final test accuracies cluster within seed noise. We retain the experiment in the appendix because (i) it verifies the paired-seed protocol on a small example, (ii) it reproduces Vahedi and Ilies [20]’s trajectory-visualisation style on a neural network rather than a 2-D test function, and (iii) the resulting picture exposes a real geometric difference between adaptive and non-adaptive optimisers in weight space. The head-to-head numbers reported in the main paper come from Exps 4.1, 4.3, and 4.4.

**Setup.** A  $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$  MLP is trained on the `make_moons` dataset (1,000 points, noise 0.2) full-batch for 2,000 steps. Five optimisers, five seeds, 25 runs total. Following Vahedi and Ilies [20]’s visualisation style, seed 0 of each optimiser additionally records its weight trajectory at every 10th step; we render the loss landscape in a *shared* top-2 PCA basis (union of all five trajectories’ displacements from  $\theta_0$ ) with each optimiser’s trajectory overlaid. Centering on  $\theta_0$  and sharing the basis means PC1 and PC2 mean the same thing in every panel and the paired-seed initialisation lands at (0,0) everywhere.

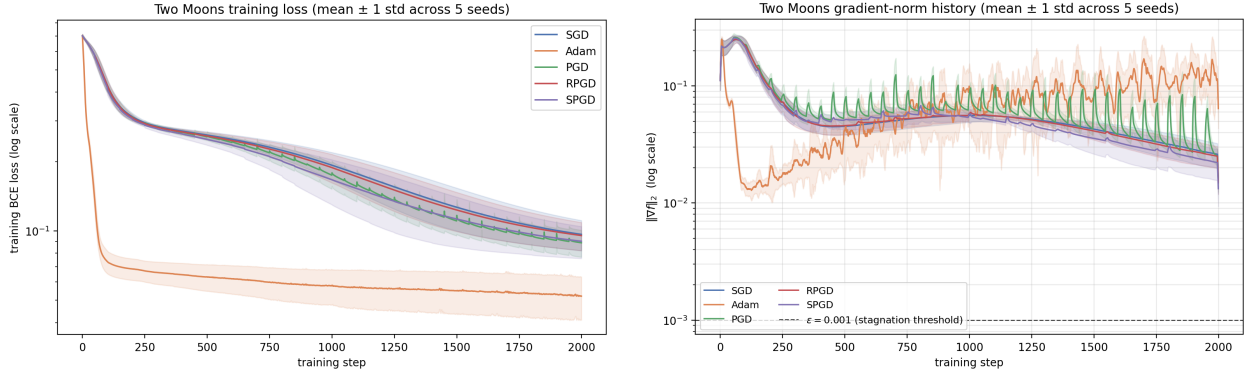
**Results — numbers.** Final test accuracies (mean  $\pm 1$  std across 5 seeds): Adam  $0.977 \pm 0.010$ , PGD  $0.970 \pm 0.018$ , SPGD  $0.970 \pm 0.015$ , RPGD  $0.967 \pm 0.019$ , SGD  $0.966 \pm 0.019$ . The four gradient methods are within seed noise of each other; Adam wins for the well-known reason that adaptive scaling fits a benign basin quickly. This experiment does not adjudicate SPGD vs. baselines.

**Results — trajectories.** Figure 10 is the visualisation showpiece. Two features are robust across seeds: (i) the four gradient methods (SGD, PGD, RPGD, SPGD) all move predominantly along the shared PC2 direction, while Adam moves substantially along PC1 — a direct picture of adaptive scaling exploring directions the gradient methods do not preferentially use; (ii) SPGD’s path is faintly serrated relative to SGD’s, the small lateral excursions are SPGD’s accepted perturbation candidates. These are diagnostic observations about *geometry*, not claims about *which optimiser wins*.

Figure 11a shows training-loss curves and Figure 11b the gradient-norm history with the proposal’s stagnation threshold  $\varepsilon = 10^{-4}$  drawn for reference. On this dataset the loss landscape is sufficiently benign that *none* of the five optimisers exhibits sustained stagnation ( $\|\nabla f\| < 10^{-4}$  for  $\geq 5$  consecutive steps); the “saddle escape” mechanism is therefore not observable in the endpoint statistics here, only as the small trajectory feature noted above.



**Figure 10:** Two Moons training-BCE loss surface in a *shared* PCA basis (seed 0). PC1 and PC2 are the top-2 principal directions of the union of weight displacements  $\theta(t) - \theta_0$  across all five optimisers, so every panel projects onto the same 2-D plane through 1185-D weight space. The colour map and white iso-loss contours show the training BCE loss evaluated at  $\theta_0 + a\mathbf{u} + b\mathbf{v}$ , clipped to the loss range the trajectories actually visit (yellow = approaching the clip ceiling, purple = low-loss basin). Contour levels are log-spaced so the basin’s structure is legible; an unclipped colour scale would be dominated by the saturation strip on the left edge, where weights at the boundary of the sweep grid push the untrained-network BCE loss to  $\sim 50$ . Green circle = shared initialisation  $\theta_0$  at the origin, red “X” = final point, white curve = weight trajectory. The geometric reading: *the four gradient methods all move predominantly along PC2, whereas Adam’s trajectory has substantial PC1 motion* — per-parameter adaptive scaling moves the iterate along a direction the gradient methods do not preferentially explore. SPGD’s white path is faintly serrated relative to SGD’s: those tiny lateral excursions are SPGD’s perturbation phases.



(a) Training BCE loss (mean  $\pm 1$  std across 5 seeds, log scale). (b) Gradient-norm trajectories with the proposal’s stagnation threshold  $\varepsilon = 10^{-4}$  drawn as a dashed line.

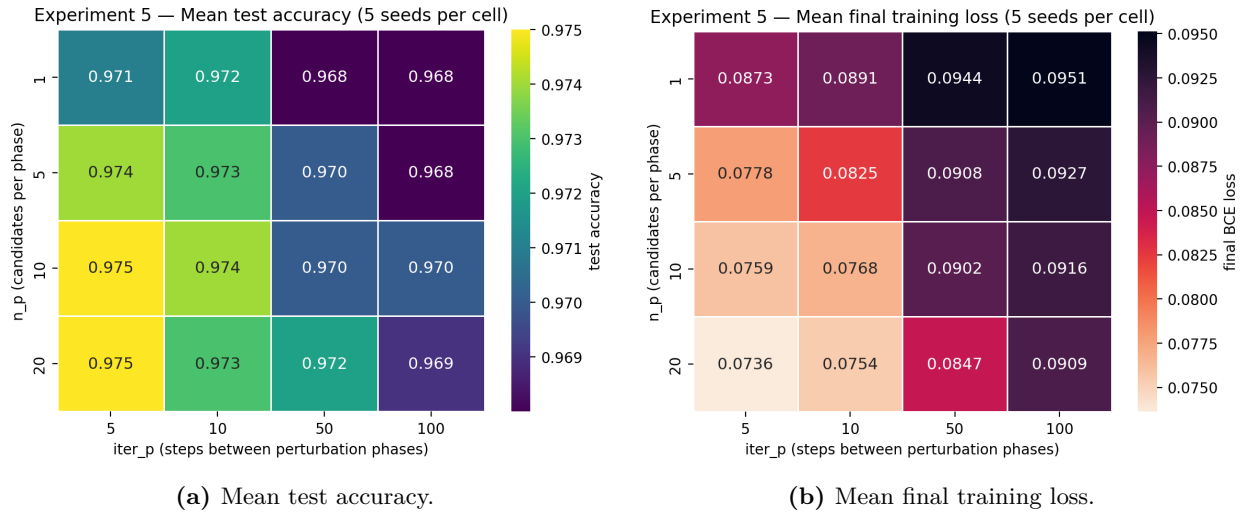
**Figure 11:** Two Moons summary. Left: training loss; Adam pulls away early. Right:  $\|\nabla f\|_2$  vs. training step on log- $y$  — no optimiser’s gradient norm dips below the stagnation threshold, which is why the stagnation episode count is zero across all 25 runs.

## B $N_P \times \text{Iter}P$ Hyperparameter Ablation (proposal Exp 5)

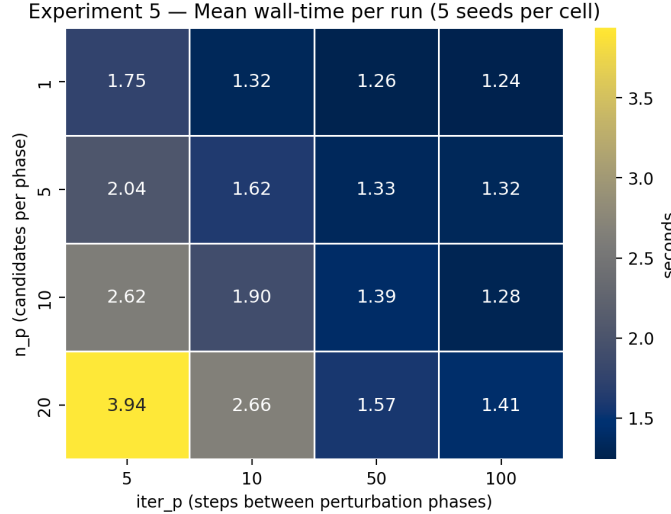
This appendix reports SPGD’s sensitivity to its two perturbation hyperparameters. The ablation runs on the same Two Moons setup as Appendix A. Because that landscape is non-discriminating (zero stagnation across all 25 main-experiment runs) the ablation similarly does not produce a saddle-escape signal — every cell of the grid sees zero stagnation episodes — and is consequently a hyperparameter-robustness diagnostic rather than a discriminating benchmark. We retain it because the *compute-accuracy trade-off* it reveals is independent of saddle structure.

**Setup.** Grid:  $N_P \in \{1, 5, 10, 20\}$  and  $\text{Iter}P \in \{5, 10, 50, 100\}$ , five seeds per cell, totalling 80 runs. Total candidate forward passes per run =  $(n_{\text{steps}}/\text{Iter}P) \cdot N_P$ , which spans roughly 20 to 8000 across the grid (a  $\sim 400\times$  compute range with all other hyperparameters fixed).

**Results.** Figure 12 reports the mean test accuracy and final-training-loss heatmaps; Figure 13 shows the corresponding wall-time heatmap. Mean stagnation episodes are zero across the entire grid, so the stagnation heatmap is omitted as uninformative. The accuracy heatmap is largely flat: SPGD on Two Moons is robust to the precise choice of  $N_P$  and  $\text{Iter}P$  within reasonable bounds. The final-loss heatmap shows a faint trend toward smaller  $\text{Iter}P$  (more frequent perturbation phases) reaching slightly lower training loss without a corresponding accuracy gain, consistent with SPGD finding broader basins rather than deeper ones. The cheapest cell within 1% of the best mean test accuracy identifies a practical operating regime that trades essentially no accuracy for a substantial reduction in candidate forward-pass cost. The wall-time heatmap follows the predicted  $(n_{\text{steps}}/\text{Iter}P) \cdot N_P$  scaling, so when SPGD is deployed in a compute-budgeted setting, choosing  $N_P$  and  $\text{Iter}P$  at the cheap-but-not-worst-accuracy corner of the heatmap is the natural recommendation.



**Figure 12:** Ablation heatmaps. Left: test accuracy is largely insensitive to  $N_P$  and  $\text{Iter}P$  within the explored range, suggesting SPGD’s hyperparameter choice is forgiving on benign landscapes. Right: final BCE training loss exposes a small but visible compute-quality trade-off — smaller  $\text{Iter}P$  reaches slightly lower training loss — which the accuracy view alone cannot resolve.



**Figure 13:** Per-run wall time across the  $N_P \times IterP$  grid. The pattern follows the analytic prediction  $\propto (n_{\text{steps}}/IterP) \cdot N_P$ . Choosing the smallest  $N_P$  and largest  $IterP$  that does not sacrifice accuracy is the practical recipe for cost-effective SPGD.

## Acknowledgments

The author thanks the WPI Academic Computing team for access to the Turing cluster, on which the CIFAR-10 anchor experiment was carried out. This work was completed in fulfillment of the final-project requirement of MA 551 (Computational Statistics) at Worcester Polytechnic Institute.

## References

- [1] Bernd Bischl, Giuseppe Casalicchio, Matthias Feurer, Pieter Gijsbers, Frank Hutter, Michel Lang, Rafael Gomes Mantovani, Jan N. van Rijn, and Joaquin Vanschoren. OpenML benchmarking suites. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [2] Samuel Burer and Renato D. C. Monteiro. A nonlinear programming algorithm for solving semidefinite programs via low-rank factorization. *Mathematical Programming*, 95(2):329–357, 2003.
- [3] Anna Choromanska, Mikael Henaff, Michaël Mathieu, Gérard Ben Arous, and Yann LeCun. The loss surfaces of multilayer networks. *Journal of Machine Learning Research — Proceedings of AISTATS*, 38:192–204, 2015.
- [4] Yann N. Dauphin, Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, Surya Ganguli, and Yoshua Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014.
- [5] Rong Ge, Furong Huang, Chi Jin, and Yang Yuan. Escaping from saddle points — online stochastic gradient for tensor decomposition. In *Conference on Learning Theory (COLT)*, pages 797–842, 2015.
- [6] Rong Ge, Jason D. Lee, and Tengyu Ma. Matrix completion has no spurious local minimum. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.

- [7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL <http://www.deeplearningbook.org>.
- [8] F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4):1–19, 2015.
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [11] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.
- [12] Chi Jin, Rong Ge, Praneeth Netrapalli, Sham M. Kakade, and Michael I. Jordan. How to escape saddle points efficiently. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70, pages 1724–1732. PMLR, 2017.
- [13] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015.
- [14] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [15] Jason D. Lee, Max Simchowitz, Michael I. Jordan, and Benjamin Recht. Gradient descent only converges to minimizers. In *Conference on Learning Theory (COLT)*, pages 1246–1257, 2016.
- [16] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- [17] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [18] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [19] Ju Sun, Qing Qu, and John Wright. A geometric analysis of phase retrieval. *Foundations of Computational Mathematics*, 18(5):1131–1198, 2018.
- [20] Amir M. Vahedi and Horea T. Ilies. SPGD: Steepest perturbed gradient descent optimization. *arXiv preprint arXiv:2411.04946*, 2024. URL <https://arxiv.org/abs/2411.04946>.
- [21] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. OpenML: Networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, 2014.